Project Documentation

C64 Keyboard: Software

Project number: 231

Revision: 0

Date: 09.08.2026

C64 Keyboard Software

Module Description v0.00

Table of Contents

1. Introduction

The C64 Keyboard functions as a normal/standard matrix keyboard if no micro controller (Pro Micro) is installed. However, if you want to use "smart" functions, like

- RESET/EXROM RESET
- Kernal Switching
- RGB LED illumination (WS2812B strip)
- USB Keyboard (for VICE or BMC64)

some software needs to be installed on the Pro Micro. This can be done using the free Arduino IDE, a suitable USB-C or micro-USB cable, and a PC.

The software can be configured to suit your particular use case simply by uncommenting the appropriate preprocessor macros. This does not require a PhD in computer science and will be described later in this document.

2. Functions

2.1. C64 (Matrix) keyboard

This requires the Kernal switcher cable to be connected to a Kernal EPROM adapter (long board or short board), plus /RESET and /EXROM.

Key	Duration	Function
RESTORE + RESET	< 0.3 secs	Normal RESTORE
RESTORE + RESET	≥ 0.3 secs	"RESTORE Fix": 5 pulses (of 30 msecs each) are issued on the RESTORE signal of the keyboard.
RESTORE + RESET	≥ 3 secs	A "normal" RESET is issued. SHIFT LOCK is turned off.
RESTORE + RESET	≥ 5 secs	An EXROM RESET is issued. This bypasses some CBM80 copy protections. SHIFT LOCK is turned off.
PREV		Cycle through RGB LED modes (downwards)
NEXT		Cycle through RGB LED modes (upwards)
PREV + RESTORE + RESET		Cycle through Kernals (downwards), RESET the C64
NEXT + RESTORE + RESET		Cycle through Kernals (upwards), RESET the C64

The requirement to press **PREV/NEXT** and **RESTORE** simultaneously prevents accidentally switching Kernals, since doing so resets the C64 and causes any running program to be lost.

2.2. Ultimate (Matrix) keyboard

This requires the **Ultimate Buttons cable** to be connected to the **Buttons** pin header on the **Ultimate Elite II** board. On earlier models, the button pin header is not implemented.

Key	Duration	Function
RESTORE / RESET	< 0.3 secs	RESTORE key
RESTORE / RESET	≥ 0.3 secs	RESTORE key
RESTORE / RESET	≥ 3 secs	The RESET button on the buttons pin header is simulated. SHIFT LOCK is turned off.
RESTORE / RESET	≥ 5 secs	SHIFT LOCK is turned off.
PREV FREEZE		Cycle through RGB LED modes (downwards)
NEXT MENU		Cycle through RGB LED modes (upwards)
PREV FREEZE + RESTORE / RESET		The FREEZE button on the buttons pin header is simulated.*
NEXT MENU + RESTORE / RESET		The MENU button on the buttons pin header is simulated.

* The **FREEZE** button only has an effect when a freeze cartridge is present.

2.3. VICE USB Keyboard

The USB keyboard in **VICE mode** uses a positional keyboard layout in either **English, German, or Swedish**. The keyboard is scanned 125 times per second (every 8 ms).

Key	Duration	Function
RESTORE / RESET	< 0.3 secs	RESTORE key sent.
RESTORE / RESET	≥ 0.3 secs	RESTORE key sent twice.
RESTORE / RESET	≥ 3 secs	A soft RESET (ALT+F9) is sent. SHIFT LOCK is turned off.
RESTORE / RESET	≥ 5 secs	A hard RESET (ALT+F12) is sent. SHIFT LOCK is turned off.
PREV		Cycle through RGB LED modes (downwards)
NEXT		Cycle through RGB LED modes (upwards)
PREV + RESTORE / RESET		<ALT>+<8> is sent (attach disk image)
NEXT + RESTORE / RESET		<ALT>+<C> is sent (attach cartridge image)

2.4. BMC64 USB Keyboard

The USB keyboard in **VICE mode** is a **positional English keyboard**. The keyboard is scanned 125 times per second (every 8 ms).

Key	Duration	Function
RESTORE / RESET	< 3 secs	RESTORE key sent
RESTORE / RESET	≥ 3 secs	C= + f1 is sent. SHIFT LOCK is turned off. *
RESTORE / RESET	≥ 5 secs	C= + f3 is sent. SHIFT LOCK is turned off. *
PREV		Cycle through RGB LED modes (downwards)
NEXT		Cycle through RGB LED modes (upwards)
PREV + RESTORE / RESET		C= + f5 is sent.*
NEXT + RESTORE / RESET		C= + f7 is sent.*

* These functions require configuration in the **BMC64** menu, which can be accessed by pressing **RESTORE**. The relevant submenu is:

Keyboard → Mapping → Positional

My Configuration:

Key	Configured Actions
C= + f1	Soft reset
C= + f3	Hard reset
C= + f5	Tape OSD
C= + f7	Menu

Figure 1: Configuration of BMC64 in the menu

3. Installing and *preparing* the Arduino IDE

3.1. Download and Installation

The Arduino IDE is required for programming the Pro Micro module. In addition, several libraries need to be installed using the IDE's Library Manager.

The Arduino IDE can be downloaded from arduino.cc. Select the version for your computer's operating system (Windows, Linux, or macOS) and follow the installation instructions.

3.2. Library Manager

Next, open the **Library Manager** in the Arduino IDE.

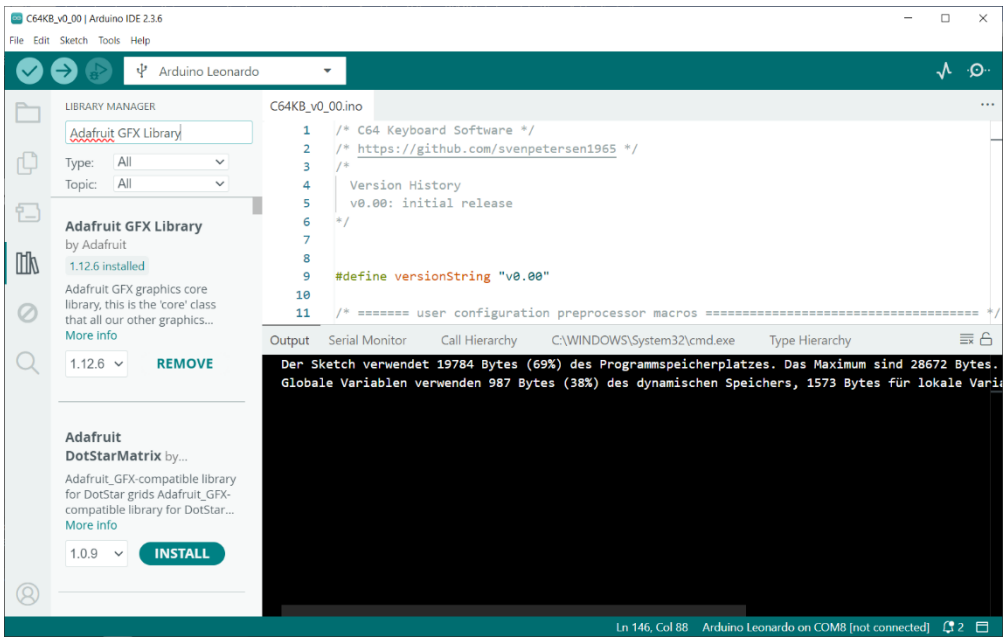

Figure 2: Library Manager (Arduino IDE 2.3.6)

The **Library Manager** can be found in the left-hand sidebar. It is represented by the **"books" icon**, as shown in Figure 2.

You need to install the following libraries:

Library	Source	Version used
Adafruit GFX Library	https://github.com/adafruit/Adafruit-GFX-Library	1.12.6
Adafruit SSD1306	https://github.com/adafruit/Adafruit_SSD1306	2.5.17
Adafruit SH110X	https://github.com/adafruit/Adafruit_SH110X	2.1.15
PCF8575	https://github.com/RobTillaart/PCF8575	0.3.0
HID-Project	https://github.com/NicoHood/HID	2.8.4
Adafruit_NeoPixel	https://github.com/adafruit/Adafruit_NeoPixel	1.15.5

The table above lists the required libraries, their Internet sources (which you probably will not need), and the library versions used for this project.

In most cases, using a newer version should not cause any problems. However, there may be cases where a newer version is incompatible with the project. If this happens, please install the specific library version used for this project.

4. Configuring the Software

4.1. Preprocessor macros

The C64 keyboard **"C64KB_v?_??.ino"** source code can be configured to suit your needs using **preprocessor macros**. These macros are evaluated during compilation and determine how the software is compiled.

Most of these macros are either **defined** or **undefined**. This allows options to be switched **on or off** simply by uncommenting or commenting out the corresponding line.

Comments are text annotations that are not compiled. A single-line comment starts with `//`. Preprocessor macros themselves are defined using a `#define` statement.

Note: Comments can also be enclosed between `/*` and `*/`. However, this is less convenient when working with preprocessor macros.

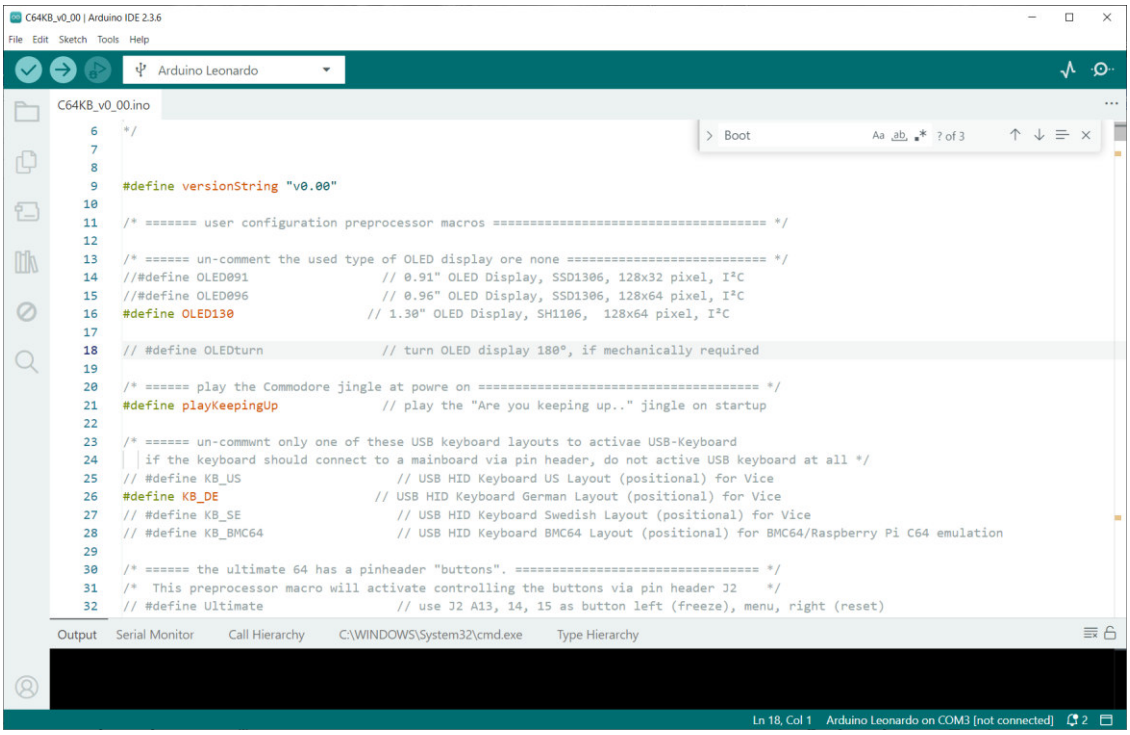

Figure 3: Some preprocessor macros (Arduino IDE)

In Figure 3 the preprocessor macros `OLED130`, `playKeepingUp` and `KB_DE` are enabled, while all other macros are disabled (i.e., commented out).

In the Arduino IDE editor, comments are displayed in grey, while active preprocessor macros are shown in color. The meaning of these macros is explained later in this document.

4.2. OLED display

The software supports three types of OLED displays.:

Preprocessor Macro	OLED Type
`OLED091`	0.91" OLED Display, SSD1306 controller, 128x32 pixel, I²C
`OLED096`	0.96" OLED Display, SSD1306 controller, 128x64 pixel, I²C
`OLED130`	1.30" OLED Display, SH1106 controller, 128x64 pixel, I²C

These OLED displays are widely available on eBay, Amazon, AliExpress, and other online retailers.

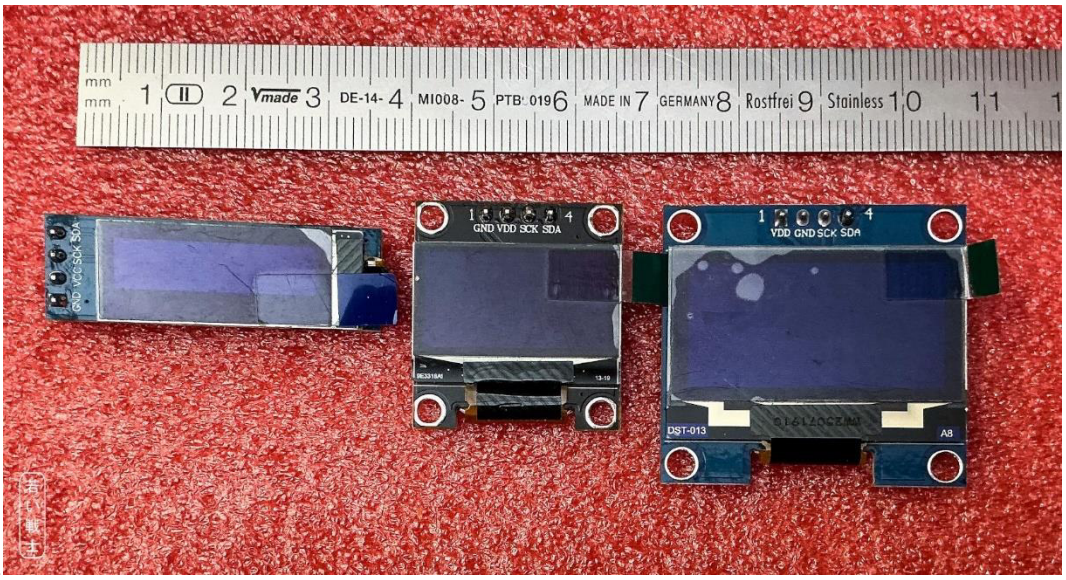

Figure 4: 0.91", 0.96" and 1.3" OLED Displays (l.t.r.)

If the display needs to be rotated 180° for a better installation, enable the `OLEDturn` preprocessor macro.

Only one display should be enabled at a time. If no display is connected, do not enable any of the display options.

Note:

- **Enabling** = uncommenting the macro definition by removing `//`.
- **Disabling** = commenting out the macro definition by adding `//` at the beginning of the line.

4.3. Playing the "Are you keeping up with the Commodore" jingle

Yes, I admit, it is a bit nerdy. I did it because I like to play…

The keyboard can play the jingle from the **1983 advertisement**. Not in its full length, though—just a few seconds.

This feature can be activated by uncommenting the `playKeepingUp`. preprocessor macro definition. If it is not enabled, the keyboard starts up with just a beep.

4.4. USB Keyboard

There are four different types of positional USB keyboards: three for the **VICE emulator** and one for **BMC64**. To use these keyboards, VICE or BMC64 must be configured accordingly.

Preprocessor Macro	Keyboard Layout
`KB_US`	VICE, positional US Layout
`KB_DE`	VICE, positional German Layout
`KB_SE`	VICE, positional Swedish Layout
`KB_BMC64`	BMC64, positional US Layout

The software does not support **symbolic keyboard layouts** (yet).

Only activate one or none of these layouts. If you want to connect the keyboard to a C64 or an Ultimate 64 via the pin header, do not enable any USB keyboard layout, as the keyboard scanning will interfere with the computer's keyboard scanning.

The **Ultimate 64** and **C64 Ultimate** use a symbolic keyboard layout and are therefore not supported. The **TheC64 Mini** also uses a symbolic keyboard layout and is not supported. I was unable to determine which keyboard modes are required by the TheC64 Mini.

The USB keyboard requires the **GPIO/PCF8575 module** to scan the keyboard matrix.

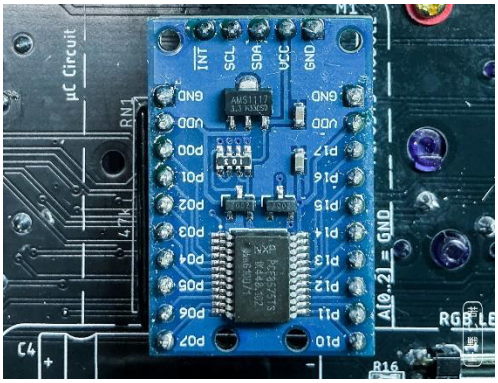

Figure 5: PCF8575 module M1

4.5. Ultimate 64 mode

In this mode, the keyboard is connected to the **Ultimate 64 Elite II** or **C64 Ultimate** mainboard via the 20-way ribbon cable/pin header. In addition, a cable must be connected between **J2** and the **"Buttons"** pin header on the mainboard. In **Ultimate mode**, these buttons are emulated.

To enable this software option, uncomment the `Ultimate` preprocessor macro definition.

If you want to use the **Kernal Switch** functions with a **regular C64** mainboard, disable this option.

4.6. WS2812B RGB LED Strip

The RGB LED strip can be enabled by uncommenting the `RGB` preprocessor macro definition. This requires several additional parameters, which are also defined as preprocessor macros:

- `numRGB`: The number of LEDs on your strip.
- `effectLen`: The length of the effect. It should be ≥ **numRGB** and can be a multiple of **numRGB**.
- `RGBbright`: The brightness of the LED effect.
- `RGBspeed`: The speed of the fade effect.

If `numRGB` is smaller than the actual number of LEDs, not all LEDs will be used. If it is larger, the additional value only wastes memory.

The brightness (and, in some cases, the hue) of the individual LEDs is derived from a sine wave function. Its wavelength is determined by `effectLen`. This value can—and should—be longer than the total number of LEDs (`numRGB`), resulting in a smoother and more subtle lighting effect.

`RGBbright` determines the brightness of the lighting effect—the amplitude of the sine wave described above. The maximum value is **255**, while **0** turns the LEDs off. RGB LEDs can be very bright, potentially becoming distracting or even uncomfortable. Reflections from the LEDs on the monitor can also reduce the enjoyment of using the system. Therefore, the brightness should be selected carefully. I also recommend avoiding positioning the LEDs so that they point directly toward the monitor or the user's face.

`RGBspeed` determines the speed of the lighting effect. **0** is the fastest setting and the recommended value.

4.7. Kernal Names

The OLED display shows the Kernal name when the C64 Keyboard is used as a Kernal switcher. There are eight Kernals, or seven when using a short-board Kernal adapter. The default names are **"Kernal #1"** through **"Kernal #8"**. These names can be customized to match the Kernals programmed onto the Kernal EPROM. The names should not exceed **9 characters** in length.

The names are defined using the preprocessor macros `KrnName1` through `KrnName8`.

4.8. Boot Logo

There are two logos that can be shown on the OLED display during startup: the **C64 logo** and the **VIC-20 logo**, since the keyboard also works with the VIC-20.

Figure 6: 64 vs. VIC-20 logo

The `Logo64` preprocessor macro displays the C64 logo, while `Logo20` displays the VIC-20 logo. If neither preprocessor macro is defined, only the software version string (`versionString`) is displayed.

5. Programming The Software

The software is programmed using the **Arduino IDE**. All you need is a suitable USB cable: either a **micro-USB** or **USB-C** cable, depending on the model of your Pro Micro.

If you have a micro-USB model, please be careful while the cable is connected. The micro-USB connector is mounted directly on the PCB and can break off easily. **USB-C Pro Micro** boards are considerably more robust and are therefore the recommended model.

Connect the Pro Micro to your computer. It can either be connected to the **C64 Keyboard** or directly to the USB cable.

First, make sure that you select the correct board type in the **Tools → Board → Arduino AVR Boards** menu, as shown in Figure 7.

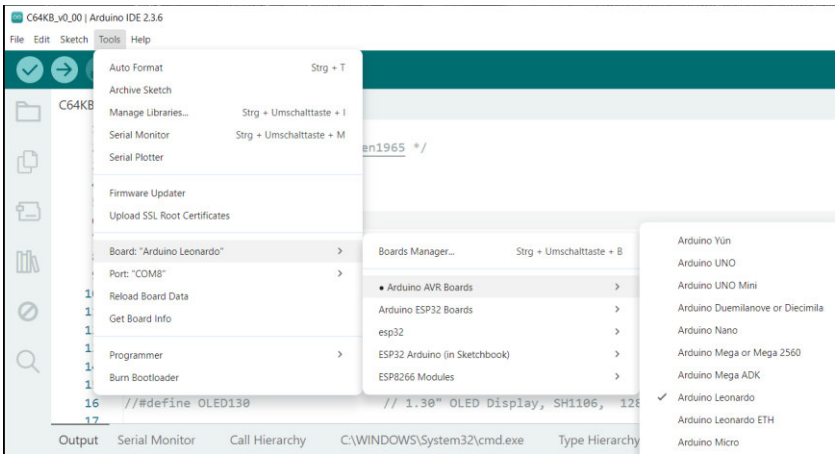

Figure 7: Selecting Arduino Leonardo

Now, select the appropriate **COM port** from the **Tools → Port** menu.

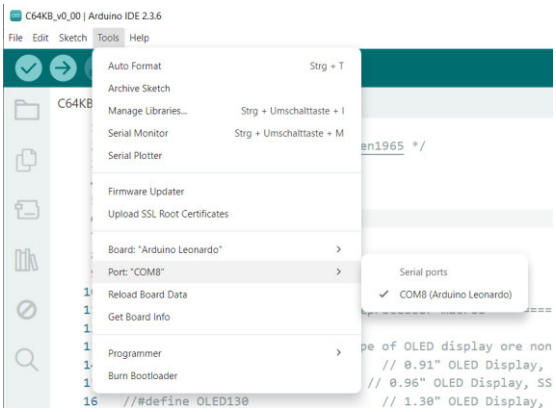

Figure 8: Selecting the port.

In my case, it is **COM8**. The port number depends on how the Pro Micro is registered on your computer, so it will be different for each system.

If you cannot find the **Arduino Leonardo** port, check the **Device Manager** in Windows:

Windows → Device Manager → Ports (COM & LPT)

The connected Pro Micro should appear there as **Arduino Leonardo**.

There are three buttons in the teal toolbar at the top of the Arduino IDE: Verify (✓), Upload (→) and Debug (▸).

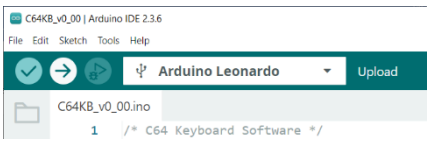

Figure 9: Verify, Upload and Debug buttons

The Verify button compiles the sketch (the term used by Arduino for source code). The Upload button also compiles the sketch, then connects to the Pro Micro and uploads the compiled program.

The result of compiling the source code (C64KB_v0_00.ino, where v0_00 indicates the version and may change) is displayed in the Output window at the bottom of the Arduino IDE. The output should show how much of the Pro Micro's flash memory and RAM is being used, without any error messages.

At some point during the upload process, the Arduino IDE will wait for the Upload Port. This should not take very long. The avrdude software will then write the program to the Pro Micro's flash memory. It produces red-colored output in the same window.

```
Using port            : COM7
Using programmer      : avr109
Setting baud rate     : 57600
AVR part              : ATmega32U4
Programming modes     : SPM, ISP, HVPP, JTAG
Programmer type       : butterfly
Description           : Atmel bootloader (AVR109, AVR911)
connecting to programmer: .
Programmer id    = CATERIN; type = S
Software version = 1.0; no hardware version given
programmer supports auto addr increment
programmer supports buffered memory access with buffersize=128 bytes
Devcode selected: 0x44

AVR device initialized and ready to accept instructions
Device signature = 1E 95 87 (ATmega32U4)
Reading 21036 bytes for flash from input file C64KB_v0_00.ino.hex
in 1 section [0, 0x522b]: 165 pages and 84 pad bytes
Writing 21036 bytes to flash
Writing | ################################################## | 100% 1.62s
21036 bytes of flash written
Avrdude done.  Thank you.
```

Figure 10: avrdude

It is important that the Writing | ## | progress indicator reaches **100%**. This confirms that the Pro Micro has been programmed successfully.

In some cases, the Arduino IDE may fail to connect to the Pro Micro and remain at **"Waiting for upload port..."** for an extended period. I have read that this can be caused by incorrectly configured USB flags.

This problem can be resolved by resetting the Pro Micro after **"Waiting for upload port..."** has been displayed twice. To do this, briefly short the adjacent **RST** and **GND** pins on the Pro Micro using a jumper (Figure 11) or a reset button (Figure 12).

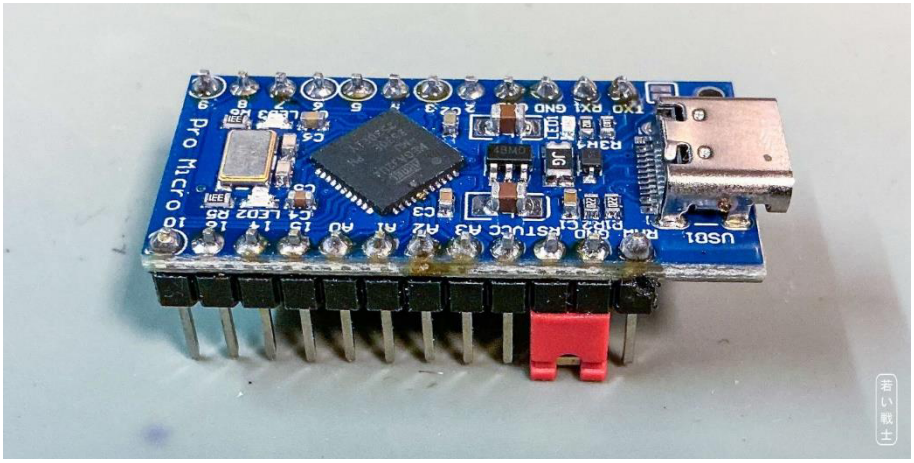

Figure 11: Pro Micro with a Reset jumper

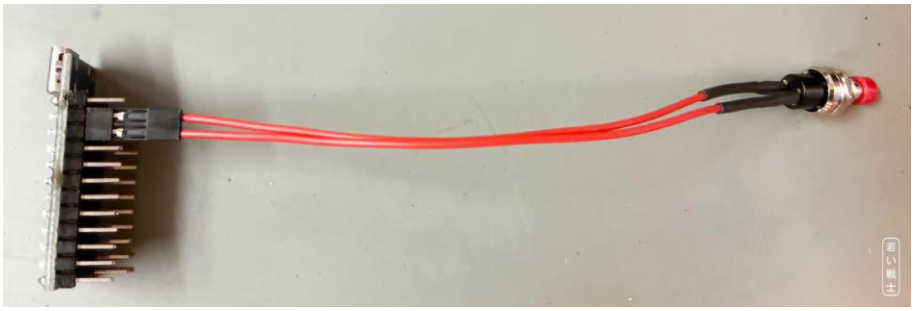

Figure 12: Pro Micro with a Reset button

In most cases, resetting the Pro Micro once was sufficient. In some cases, however, it was necessary to press the reset button **twice**. The same procedure can be performed using a jumper, or by briefly shorting the **RST** and **GND** pins with another suitable tool, such as tweezers or a Dupont wire.

6. Revision History

6.1. Rev. 0.00

- Initial Release